BÁO CÁO KỸ THUẬT

Distributed Streaming-based Client-Server Collaboration Application

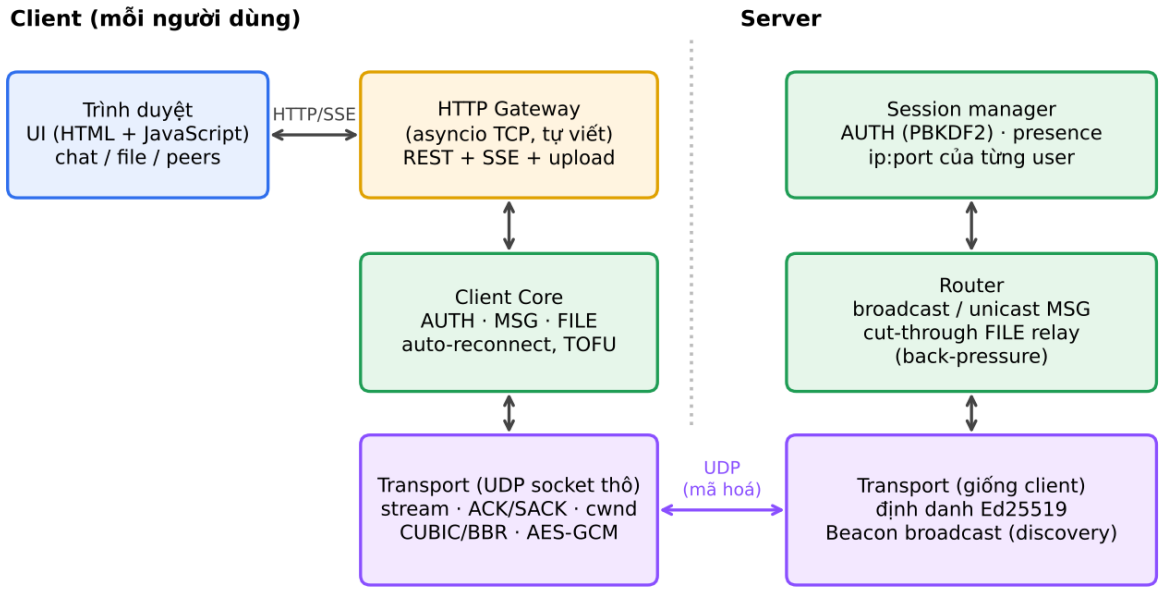
Môn: Network Application Development · Assignment 1

Hệ thống chat, phát hiện người dùng và truyền file thời gian thực chạy trên UDP thuần, với transport tin cậy tự xây dựng (đa stream, sliding window, ACK/SACK, retransmit chọn lọc, CUBIC/BBR) và mã hoá AES-256-GCM. Giao diện web bằng HTTP + JavaScript phía client.

Hạng mục	Nội dung
Ngôn ngữ / thư viện	Python 3.10+; chỉ dùng socket thô + asyncio; 1 thư viện ngoài: cryptography (AES-GCM, X25519, Ed25519, HKDF)
Không dùng	Flask/Django, WebSocket, QUIC/aioquic, thư viện mạng cấp cao
Giao thức	Giao thức tầng ứng dụng + transport tự thiết kế (xem mục 2)
Kiểm thử	19 test tự động (unit, transport, e2e, gateway) + kiểm thử trình duyệt thật bằng Chromium

1. Kiến trúc hệ thống

Hệ thống tách bạch ba lớp: logic mạng (common/transport.py, client/core.py, server/server.py), xử lý dữ liệu (protocol.py, crypto.py) và giao diện (client/web: HTML + JS thuần). Trình duyệt chỉ nói chuyện HTTP với gateway cục bộ; mọi giao tiếp giữa các máy đều là UDP.



Nhiều stream độc lập trong 1 kết nối UDP: ctrl · mỗi tin nhắn 1 stream · mỗi file 1 stream

Quyết định thiết kế chính

Vấn đề	Giải pháp	Liên hệ lý thuyết (giáo trình)
Tin cậy trên UDP	Số hiệu gói tăng đơn điệu (mỗi lần gửi lại dùng số mới), ACK dạng dải (SACK), phát hiện mất gói bằng ngưỡng 3 gói + ngưỡng thời gian 9/8 RTT + PTO	Ch.3: rdt, Go-Back-N / Selective Repeat, TCP RTT
Không bị Head-of-Line blocking	Mỗi tin nhắn / mỗi file là một stream riêng; chỉ byte trong cùng stream mới phải đúng thứ tự	Ch.2: HTTP/2→HTTP/3, QUIC
Điều khiển luồng	Cửa sổ nhận trượt theo từng stream (MAX_STREAM_DATA), 512 KB; server giữ credit khi người nhận chậm (back-pressure)	Ch.3: flow control, rwnd
Điều khiển tắc nghẽn	CUBIC (mặc định) hoặc BBR (--cc bbr) + pacing	Ch.3: AIMD, CUBIC, BBR
Bảo mật	X25519 tạm thời (forward secrecy) → HKDF → 2 khoá AES-256-GCM; server ký handshake bằng Ed25519, client ghim fingerprint (TOFU); header là AAD; cửa sổ chống replay	Ch.8: DH, chữ ký số, MITM, TLS 1.3, IPsec replay window
Phát hiện / presence	Server broadcast beacon UDP; danh sách peers (user, ip, port) do server duy trì và đẩy mỗi khi thay đổi	Ch.2, Ch.6

2. Đặc tả giao thức

2.1 Datagram (tầng transport)

```
+------+-----------+------------------+------------------------------+---------+
| type | dest_cid  |   packet number  |  frames, mã hoá AES-256-GCM  | GCM tag |
| 1 B  |   4 B     |       8 B        |          (<= 1171 B)         |  16 B   |
+------+-----------+------------------+------------------------------+---------+
 \_____ header 13 B (được xác thực làm AAD) ____/    MTU datagram = 1200 B
```

type	Ý nghĩa	Nội dung
0x01 INITIAL	Client hello (rõ)	client_cid(4) + X25519 public(32); client gửi lại với backoff 0.25→1 s
0x02 HANDSHAKE	Server hello (rõ)	server_cid(4) + X25519 public(32) + Ed25519 public(32) + chữ ký(64) trên transcript
0x10 DATA	Gói ứng dụng	pn(8) tăng đơn điệu; nonce = $0^4 \parallel$ pn; khoá riêng cho mỗi chiều

2.2 Frame (bên trong gói DATA đã giải mã)

Frame	Trường	Mục đích
STREAM 0x02 / 0x03(FIN)	stream_id(4) offset(8) length(2) data	Dữ liệu theo byte-offset; FIN đánh dấu hết stream
ACK 0x04	largest(8) delay_us(4) n(1) n×[start(8) end(8)]	SACK: tối đa 6 dải gói đã nhận → chỉ gửi lại đoạn thiếu
MAX_STREAM_DATA 0x05	stream_id(4) max_offset(8)	Cấp thêm credit (cửa sổ trượt) cho người gửi
PING 0x06	—	Keep-alive / heartbeat mỗi 4 s khi rảnh; probe khi PTO
CLOSE 0x07	code(2) len(2) reason	Đóng kết nối có lý do (gửi 3 lần)

2.3 Thông điệp ứng dụng (u32 độ dài + JSON UTF-8, luôn có trường t)

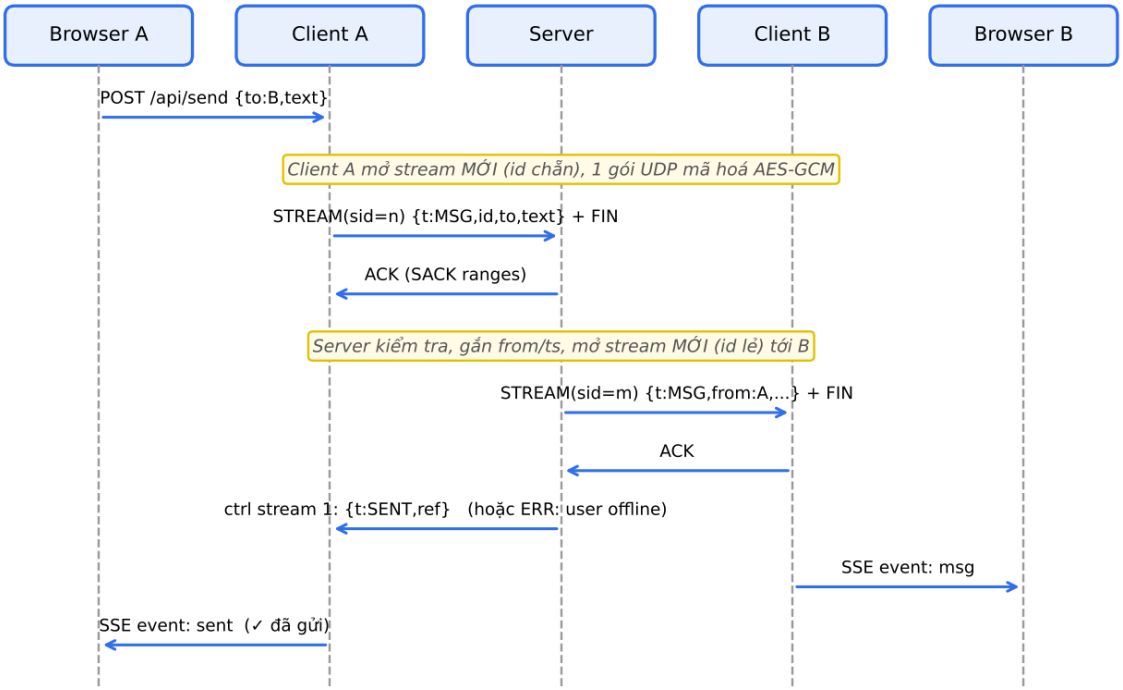
Stream	Thông điệp	Hướng	Mô tả
client #0 (ctrl)	AUTH {user,pass}	C→S	Đăng nhập; tên mới được tự đăng ký (PBKDF2-SHA256 120k vòng + salt)
client #0	LIST / BYE	C→S	Yêu cầu danh sách peers / thoát
server #1 (ctrl)	AUTH_OK {user,peers} · AUTH_FAIL {msg}	S→C	Kết quả đăng nhập
server #1	PEERS {peers:[{user,ip,port,since}]}	S→C	Cập nhật presence (khi ai đó vào/ra)
server #1	SENT {ref,n} · ERR {ref,msg}	S→C	Xác nhận đã chuyển / lỗi (user offline, file quá lớn…)
mỗi tin nhắn	MSG {id,to,text} (to = tên hoặc *)	C→S→C	Một stream mới + FIN; server thêm from, ts rồi chuyển tiếp
mỗi file	FILE {id,to,name,size} · bytes · FILE_END {sha256}	C→S→C	Một stream mới (ưu tiên thấp); server dựng lại header tin cậy và relay cut-through

2.4 Máy trạng thái kết nối

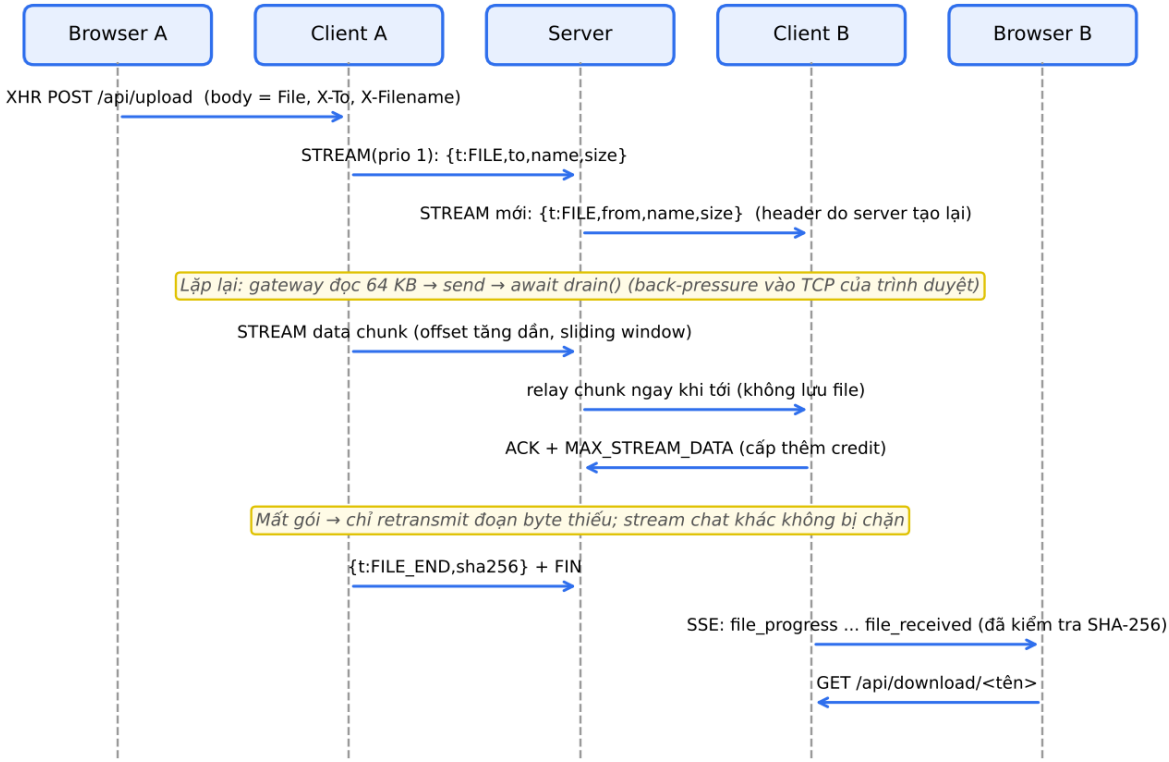
```
CLIENT                                              SERVER
CONNECTING --INITIAL(cid,epk)--------------------> (tạo Connection, cache reply)
           <-HANDSHAKE(cid',epk',pk,sig)---------
(verify sig + pin) --> ESTABLISHED <-- keys (HKDF) --> ESTABLISHED
AUTH (stream 0, mã hoá) ---------------------------> ki□m tra mật kh□u
           <-AUTH_OK / AUTH_FAIL (stream 1)-------- ONLINE  --> PEERS tới mọi người
idle > 20 s hoặc CLOSE ---------------------------> OFFLINE --> PEERS cập nhật
m□t k□t n□i b□t ngờ -> RECONNECTING (backoff 1,2,4..15 s) -> đăng nhập lại tự động
```

3. Sơ đồ tuần tự

Vòng đời một tin nhắn (A gửi unicast cho B)

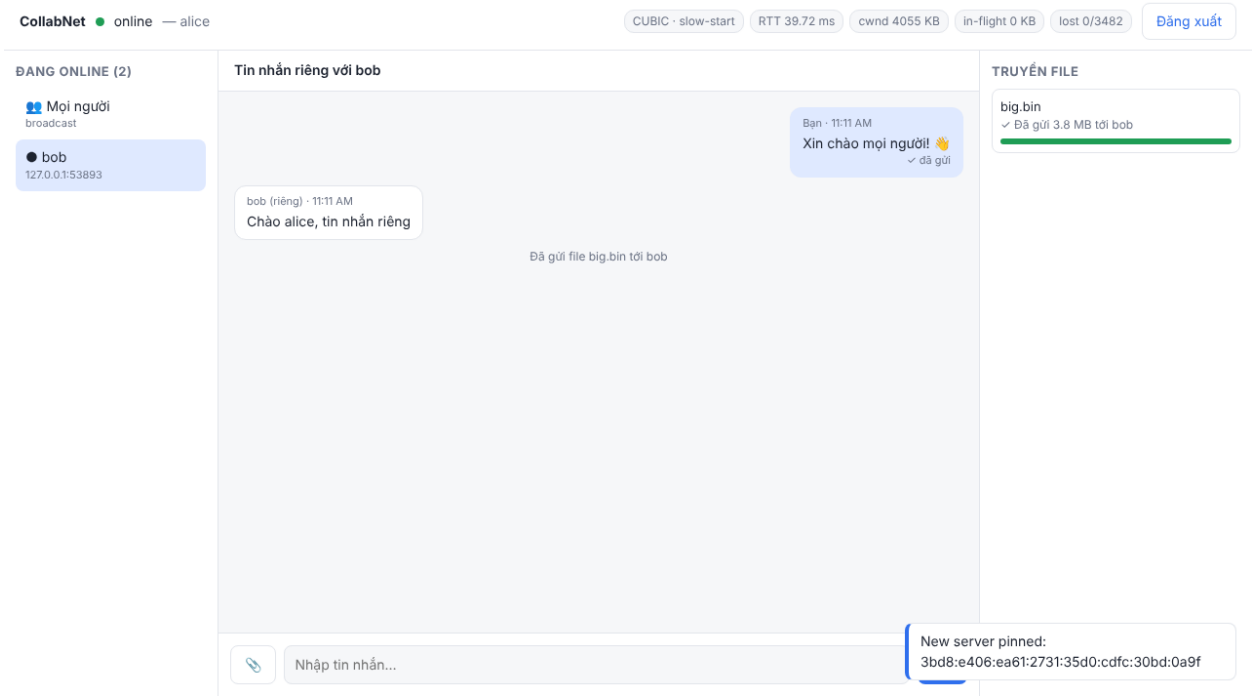


Vòng đời một lần truyền file (A → B, cut-through relay)



4. Giao diện và kết quả kiểm thử

Giao diện web (HTTP + JavaScript phía client) hiển thị tin nhắn, danh sách người dùng đang online (kèm IP:port), thanh tiến trình truyền file (gửi/nhận) và chỉ số mạng thời gian thực (thuật toán CC, RTT, cwnd, gói mất).



Nhóm test	Nội dung kiểm tra	KQ
Unit	RangeSet, mã hoá/giải mã frame, crypto (ECDH, AES-GCM chống sửa header, PBKDF2)	OK
Transport	4 MB sạch; 5% mất gói + reorder (CUBIC, BBR) – SHA-256 khớp; flow control chặn đúng cửa sổ; từ chối server sai khoá; phát hiện ngắt kết nối	OK
No-HoL	Tin chat nhỏ trong khi file 3 MB đang truyền, 3% mất gói: độ trễ ~21 ms/tin	OK
E2E (3 client)	Đăng nhập, presence, broadcast, unicast không rò rỉ, user offline→ERR, file 3 MB + file rỗng, client biến mất đột ngột, đăng nhập trùng, auto-reconnect — chạy cả trên link 4% mất gói + 15 ms (BBR)	OK
Gateway HTTP	Tĩnh/API/SSE, upload stream qua POST, tải về, chặn thiếu X-Requested-With, chặn Host lạ, chặn path traversal	OK
Trình duyệt thật	Chromium điều khiển 2 người dùng: chat, tin riêng, gửi file 4 MB, tải về đúng SHA-256, 0 lỗi JS	OK

Hiệu năng: CUBIC so với BBR (2 MB, RTT 20 ms mô phỏng)

Mất gói	CUBIC (Mbps)	BBR (Mbps)
0%	51.76	38.17
1%	6.46	37.33
3%	2.78	26.89
5%	2.43	29.17

Khi không mất gói, CUBIC nhỉnh hơn nhẹ (slow-start tích cực). Khi mất gói 1–5%, CUBIC giảm cửa sổ ×0.7 ở mỗi lần mất nên thông lượng sụp, còn BBR dựa trên mô hình băng thông/RTT nên gần như không bị ảnh hưởng — đúng với phân tích ở Ch.3 (loss-based vs model-based). Đây là lý do cho phép chọn thuật toán bằng tham số.

5. Thách thức gặp phải và cách giải quyết

Thách thức	Giải pháp
Head-of-line blocking khi chat cùng file lớn	Nếu mọi thứ nằm trong một byte-stream (như TCP) thì một gói file mất sẽ chặn cả tin nhắn. Giải pháp: transport đa stream; mỗi tin nhắn/file một stream, lập lịch ưu tiên (chat ưu tiên cao hơn file) và round-robin giữa các file. Đo được độ trễ chat ≈ 21 ms dù file đang truyền với 3% mất gói.
Mơ hồ khi đo RTT / phân biệt gửi lại	Giống TCP, ACK của gói gửi lại gây nhập nhằng. Dùng số hiệu gói mới cho mọi lần gửi (kiểu QUIC) và ghi nhớ frame nào nằm trong gói nào; khi gói bị coi là mất chỉ đưa lại dải byte tương ứng vào hàng đợi retransmit — không gửi lại cả file.
Phân mảnh (fragmentation) gói UDP	Datagram luôn ≤ 1200 B (dưới MTU) và tự phân đoạn dữ liệu thành frame STREAM, tránh phân mảnh IP — thứ làm mất cả gói khi một mảnh mất.
Deadlock do back-pressure khi relay file	Server giữ credit của người gửi khi hàng đợi gửi tới người nhận quá lớn. Lỗi thật gặp: lần giữ credit thứ hai ghi đè callback của lần trước nên credit không bao giờ được trả và luồng gửi đứng vĩnh viễn khi link mất gói. Sửa bằng danh sách callback + giải phóng toàn bộ khi người nhận đóng kết nối (phát hiện nhờ test e2e có mất gói).
Mất kết nối bất ngờ / NAT	Keep-alive PING 4 s, idle timeout 20 s ở cả hai phía; server gỡ user và phát PEERS mới; client tự kết nối lại với backoff lũy tiến và đăng nhập lại; gói hợp lệ từ địa chỉ mới sẽ cập nhật địa chỉ peer (connection migration); đăng nhập trùng sẽ đá phiên cũ.
Bảo mật trên UDP	Không có TLS sẵn: tự ghép các primitive chuẩn (X25519+HKDF+AES-GCM), ký handshake bằng Ed25519 để chống MITM (slide 8-41), ghim khoá kiểu SSH, nonce không bao giờ lặp nhờ số hiệu gói đơn điệu, cửa sổ chống replay, giới hạn số stream/credit để tránh cạn bộ nhớ.
Trình duyệt không nói UDP, và cấm framework	Viết HTTP server tối giản trên asyncio TCP (parser + SSE + upload stream). Upload dùng XHR với Content-Length, gateway đọc từng 64 KB và await drain() nên áp lực ngược truyền tới TCP của trình duyệt; chặn CSRF/DNS-rebinding bằng header riêng + kiểm tra Host.

Hạn chế đã biết: chưa có chống khuếch đại/giả mạo địa chỉ ở INITIAL (cần cookie/retry như QUIC), BBR là bản rút gọn v1, danh sách stream đã kết thúc (chống replay) lưu trong RAM, chỉ hỗ trợ IPv4.